Shwetha

Python Foundations – Activity Handouts

Activity 0.1 – Everyday Task as an Algorithm

Title: From Daily Routine to Algorithm

Instructions to Students

1. Choose one simple daily activity (examples: making tea, preparing to go to class, booking a cab).
2. Write the steps in clear, numbered points.
3. Make sure each step is:
   - Unambiguous
   - In correct order
   - Does not skip important details

Template

1. Activity chosen: Making tea
2. Write the steps:
   1. Take a vessel & pour one cup of water into it
   2. Allow the water to boil completely
   3. Add one teaspoon of tea powder to the Boiling water
   4. Add sugar in a required amount & boil for 2-3 min
   5. Add half cup of Milk to the boiling tea & turn of when it rises
   6. filter the tea into a cup
3. Check your algorithm:
   - Can a friend follow it without asking questions? Yes / No
   - If "No", refine the steps here:
     - Yes

Activity 0.2 – From Steps to Pseudo-code

Title: Writing Pseudo-code with IF and REPEAT

Instructions

1. Take your steps from Activity 0.1.
2. Identify where decisions happen (IF/ELSE).
3. Identify where repetition happens (REPEAT / WHILE).
4. Rewrite your algorithm using the following keywords:
   - IF, ELSE, ELSE IF
   - REPEAT, WHILE, UNTIL

Template

1. Original activity: Making tea
2. Pseudo-code version: S

text

START

Boil water

IF ~~Sugar~~ is required. THEN

Add ~~little~~ Sugar

ELSE

continue without sugar

ENDIF

REPEAT

Boil Mixture

UNTIL tea is ready.

END

3. Reflection:
   - Where did you use IF/ELSE? Sugar decision
   - Where did you use REPEAT/WHILE? boiling process

Activity 1.1 – Run and Edit Given Code

Title: First Python Script – Edit and Run

Instructions

1. Your teacher will give you a small Python script (example below).
2. Do NOT write new code first; only change text or numbers.
3. Run the program and observe the output.

Example Script

python

```
name = "Student"
age = 18
print("Hello,", name)
print("You are", age, "years old.")
```

Tasks

1. Change name and age to your own values.
   - name: "Shwetha"
   - age: 21
2. Run the program. What is the output?

text

Output:

Hello, Shwetha
you are, 21, years old.

3. Change the printed message to:
   - "Welcome to Python, <your name>"

Write your edited code here: ~~Sho~~ welcome to pyton
name = "Shwetha".
age = 20

python

Edited code: ~~print~~ ("welcome to pyton,"

4. Reflection:
   - What changed in the output and why?

Out put changed because we modified text & variables

Activity 1.2 – Predict–Then–Run

Title: Predict Program Output

Instructions

1. Look at each code snippet.
2. BEFORE running, write what you think the output will be.
3. Then run it and compare.

Snippet A

python

```
x = 5
y = 2
z = x + y * 3
print(z)
```

- Predicted output: 11
- Actual output: 11

Snippet B

python

```
a = 10
b = a - 3
a = a + 1
print(a, b)
```

- Predicted output: (11,7)
- Actual output: (11, 7)

Reflection

- Where was your prediction wrong? Why?

Order of operations matters.

Activity 2.1 – Calculator in Steps

Title: From Word Problem to Code

Problem 1
A student has marks in 3 subjects. Find total and average.

1. Write the steps in plain language:

   1. To find out the total marks obtained Add 3 subject marks
   2. To find out the Average of a Student
   3. Add all the 3 subject marks & Divide by the
   4. total number of subject.

2. Now convert to Python (fill in the blanks):

python

mark1 = 91

mark2 = 95

mark3 = 98

total = 284

average = 94.6

print("Total =", total)

print("Average =", average)

3. Test with your own numbers and record output.

Total = 284, Average = 94.6

Activity 2.2 – Fix the Errors

Title: Debugging Simple Python

Instructions
Each snippet has an error.

1. Write what you think is wrong.
2. Correct the code.

Snippet 1

python

print("Hello World)

- Error: It's a syntax Error, double quote is missing,
- Corrected code:

python

undefined

Snippet 2

python

def average(marks):

```
a = 10
b = "5"
c = a + b
print(c)
```

- Error: Syntax error a is integer & b is written in string.
  * integer & string can't be added
- Corrected code:

python

undefined

Reflection:

- What did you learn about syntax vs logic errors?

Syntax Error	logic Error
* It occurs when the rules is not followed in python language	* It occurs when the output will be wrong.
* programme will not run if syntax error occurs	* programme will run
* missing colon. spelling mistake are ex of syntax errors	* The errors will not be shown directly

Activity 3.1 – Rule-based Classification

Title: Grades with IF/ELSE

Rule

- If mark ≥ 90 : Grade A
- Else if mark ≥ 75 : Grade B
- Else : Grade C

Tasks

1. Draw the decision in simple text:
   - Condition 1: If Marks are ≥ 90 → grade A
   - Condition 2: If Marks are ≥ 75 & < 90 → grade B.
   - Else : Marks < 75 → grade C
2. Now write Python code:

python

```
mark = int(input("Enter mark: "))
if Mark ≥ 90 :
    print("Grade A")
elif Mark ≥ 75 :
    print("Grade B")
else:
    print("Grade C")
```

3. Test with at least 3 marks and note outputs.

i) 90 → grade A

ii) 80 → grade B

iii) 60 → grade C

Activity 3.2 – Mini Problem Bank

Title: Structured Problem Solving with IF/ELSE

For each question, fill the template.

Template

1. Restate the problem in one sentence:

check whether a given number is odd or even

2. Conditions needed (bullets):

- if number % 2 == 0 the number is even
- Else number is odd

3. Python code:

python

```
# Your solution
num = int (input ("enter the num"))
if num % 2 == 0:
  print ("even num")
Else:
  print ("odd num")
```

4. Test cases and results:

- Input: 10 Output: Even
- Input: 15 Output: Odd

Use this template for:

- Odd or even number. ✓
- Ticket discount if age < 12 or > 60.
- Check if a year is a leap year (simple rule).

Activity 4.1 – Sum with Loops

Title: Sum from 1 to N

Steps

1. In plain language:
   - What is "sum from 1 to N"? Add number from 1 to N
2. Write the algorithm:

1. Input N
2. set total = 0
3. loop from 1 to N
4. Add each Number

3. Now code it:

python

```
N = int(input("Enter N: "))
total = 0
for i in range(1, N+1):
    total = total + i
print("Sum =", total)
```

4. Try N = 5, 10, 100 and check results.

15, 55, 5050

Activity 4.2 – Star Patterns

Title: Loops for Patterns

Pattern (triangle)

```
*
**
***
****
```

1. Explain the pattern in words:

Each row increases stars

2. Write code:

python

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    print("*" * i)
```

3. Try different rows and observe.

* As the number of rows increase, the number of times increase
* each row prints stars equal to the row number
* pattern forms a triangle shape
* More rows → bigger triangle.

Activity 5.1 – Working with Lists

Title: Analyze Marks List

Given

python

marks = [56, 78, 90, 45, 88]

1. In words, write how to find:
   - Maximum mark: Max ()
   - Average mark: Sum / count
   - Count of marks ≥ 60: Loop
2. Now write Python:

python

```
marks = [56, 78, 90, 45, 88]

# Max
max_mark = Max (marks)

# Average
total = 0
for m in marks:
    total = total + m
average = total / len (marks)

# Count >= 60
count = 0
for m in marks:
    if M >= 60:
        count = count + 1

print("Max:", max_mark)
print("Average:", average)
print("Count >= 60:", count)
```

Max_Mark = 90,

* Average = 71.4
* count ≥ 60 = 3

Activity 5.2 – Choosing an Approach

Title: Which Method is Better?

Problem
Given a list of 1000 numbers, find how many are positive.

1. Approach A (loop) – describe in words:

   check each Number

2. Approach B (using built-in functions or list comprehension) – describe:

   use list comprehension

3. Which is easier to understand for you now? Why?

   (loop) because it's simple, step by step + easy for beginners.

4. Implement one approach in Python:

python

numbers = [10, -3, 5, 0, 8, -2] *# example*

 # Your code

```
Numbers = [10, -3, 5, 0, 8, -2]
count = 0
for n in numbers:
    if n > 0:
        count += 1
print ("positive numbers count:", count)
```

Activity 6.1 – Creating a Function

Title: Wrap Logic into a Function

Task

Turn average-of-marks logic into a function.

1. In words: what does the function do?

Calculate average

2. Write code:

python

```
def average_marks(marks_list):
    total = 0
    for m in marks_list:
        total = total + m
    avg = total / len(marks_list)
    return avg

data = [60, 70, 80]
print("Average =", average_marks(data))
```

3. Test with another list and note output.

data = [50, 60, 70, 80]

print("Average=", average_marks(data))

output:- Average = 65.

Activity 6.2 – Problem-Solving Template

Title: Standard Template for Any Problem

Use this for any new problem.

1. Problem understanding
   - What is given (inputs)?

     Numbers
   - What is required (outputs)?

     result
   - Any constraints (limits, ranges)?

     valid input
2. Analysis
   - Write steps in bullets (algorithm):
     - Take input Numbers, store them in a list
     - calculate total using loop, find average
     - Display result
3. Design
   - Will you use loops? Which? yes → for loop.
   - Will you use conditions (if/else)? Not required
   - Will you use functions? Which ones? yes → average_Marks()
   - What data structures (list, etc.)? List
4. Code (Python)

python

 # Your implementation

```
def average_Marks(Marks):
    total = 0
    for M in Marks:
        total += M
    avg = total / len(Marks)
    return avg
```

5. Test
   - Test case 1: input 60, 70, 80 → output 70
   - Test case 2: input 50, 60, 70 80 → output 65
   - Edge case: input 0, 0, 0 → output 0.0

Activity 7.1 – Capstone Problem

Title: Simple Fee Calculator

Problem

A class charges a base fee.

- If student is below 12 years, 50% discount.
- If above 60, 30% discount.
- Otherwise, no discount.

Use the full template from Activity 6.2 here.

(Attach a blank copy of the template and ask students to fill all five sections.)

1) problem understanding

Input = Base fee, age of student

output = Final fee after discount

Constraints = Age should be valid, fee should be positive

2) Analysis (steps)

* Input fee and age
* check age condition
* Apply discount based on age
* print final fee

3) Design:-

- functions + yes → calculate_fee()
- Data type :- float(fee), int(age)

4) code :-

```
def calculate_fee(fee, age):
    if age<12:
        fee = fee*0.5
    elif age>60:
        fee = fee*0.7
    return fee
fee = float(input("enter fee: "))
age = int(input("enter age: "))
final_fee = calculate_fee(fee, age)
print("final fee =", final_fee)
```

5) Test

- test 1 = P_1 = 1000, age = 10 → final fee = 500
- test 2 = P_2 = 1000, age = 65 → final fee = 700

Activity 7.2 – Reflection Sheet

Title: How I Solve a New Problem

1. When I see a new problem, my first step is:

   understand problem clearly & identify what is required

2. Before coding, I make sure I know:
   - Inputs: what data is given (Number, text, etc)
   - Outputs: what result is expected

3. The tools I think about (loops, if, functions) are:

   Loops, if/else conditions, and functions

4. One thing I will do differently from now on:

   I will plan the solution & write steps before coding.